TiranaFly

Designing a drone delivery network for Tirana with exact shortest paths, Set Cover and vehicle-routing heuristics on OpenStreetMap data

Case study by Iris Sinanaj, 2026

Python · OSMnx · GeoPandas · Shapely · SciPy · Matplotlib

All numbers and figures in this document are produced by the accompanying code (seed 40) and are reproducible.

Abstract

We design a drone delivery network for Tirana on real OpenStreetMap data. The city is modelled as a flight graph G = (V, E) with 1266 vertices and 103,976 edges whose vertices are 1000 clients, 60 candidate depot sites on the outskirts and 206 corners of 19 restricted-airspace polygons (hospitals, military areas, stadiums, government buildings, from OSM). Every edge is a straight flight leg that avoids restricted airspace and is no longer than the usable battery range, so Dijkstra's algorithm on G returns the exact shortest flight path between any two points (visibility-graph theorem). Depots are chosen by Set Cover over the coverage sets produced by bounded Dijkstra runs: the greedy algorithm selects 2 depots and an exact integer program certifies that 2 is optimal. Zones are the graph Voronoi cells of the depots, computed by one multi-source Dijkstra. Inside each zone, trips are built by the sweep heuristic (rotated over 12 start angles) and by Clarke-Wright savings on the exact distance matrix, improved by 2-opt, Or-opt and inter-trip relocation, and scheduled onto drones by First-Fit-Decreasing bin packing. The result for 1000 clients: 241 trips, 1285 km and 12 drones per day, against 1000 trips, 4892 km and 34 drones when every parcel flies alone. Lower bounds of 229 trips, 1110 km and 12 drones show the solution is within 5%, 16% and 0% of optimal respectively.

1. Problem Formulation

1.1 Map setup and coordinate bounds

The delivery area is the built-up part of Tirana: the union of all OpenStreetMap polygons tagged landuse = residential, business or commercial, fetched once with OSMnx and cached offline (36.49 km²). All layers are projected from WGS 84 to UTM zone 34N (EPSG:32634) so that coordinates are metres and Euclidean distances are true ground distances; plots use Web Mercator (EPSG:3857) over a cached CartoDB basemap.

1.2 Restricted airspace

Drones may not overfly hospitals, military areas, stadiums, aerodromes/heliports or government buildings. These polygons are fetched once from OSM (79 features for Tirana, among them the QSUT and Trauma university hospitals, the Ministry of Defence, the Brigades Palace and both stadiums), buffered by a 40 m horizontal safety margin, simplified and dissolved into 19 obstacle polygons with 206 vertices. Clients are never generated inside them.

1.3 Clients

1000 clients are generated by Monte-Carlo acceptance inside the coverage polygon (seed 40), each with a parcel of 0.5-2.3 kg drawn uniformly. The clients, the candidate depot sites and the obstacle vertices are the vertex set V of the flight graph.

1.4 Drone specification

The fleet drone TF-1 is a mid-size electric multicopter. The values are inside the envelope of current delivery drones (DJI FlyCart 30: 16 km loaded range; Wing and Zipline P2: 10-20 km round trips at 40-50 km/h) and are deliberately conservative for an urban environment.

Parameter	Value	Unit	Justification
Maximum flight range D_max	16	km	loaded range of a 16 km-class multicopter
Battery reserve	10	%	wind, hover, regulatory margin
Usable range D = D_max (1 - reserve)	14.4	km	feasibility threshold used everywhere
Battery constant k = 100 / D_max	6.25	% per km	linear consumption model
Maximum payload W_max	6	kg	several 0.5-2.3 kg parcels per trip
Cruise speed	50	km/h	urban cruise incl. climb and descent
Service time per client	2	min	hover and winch drop
Turnaround at depot	8	min	battery swap and loading
Delivery window	8	h/day	daylight operations

1.5 Battery model and feasibility

Battery consumption is modelled as proportional to distance, Consumption = k × Distance. A flight is feasible iff the sum of its edge lengths is at most the usable range D = 14.4 km, i.e. iff it consumes at most 90% of the battery. Hence:

- a client c can be served from depot j only if the round trip 2·dist(j, c) ≤ D, i.e. dist(j, c) ≤ D/2 = 7.2 km, where dist is the shortest-path distance in G (not the straight line);
- a multi-client cycle Depot → C■ → … → C■ → Depot is feasible iff its total shortest-path length is ≤ D and the summed parcel weight is ≤ W_max;
- any edge longer than D is removed from G (weight ∞): a leg that cannot be flown on one battery does not exist.

1.6 Formal problem

Given the flight graph G = (V, E, w), the clients C, the candidate sites S and the usable range D: (i) find the smallest S* ⊆ S such that every client has shortest-path distance ≤ D/2 to some depot in S* (minimum Set Cover); (ii) partition C into zones around the chosen depots; (iii) in each zone partition the clients into feasible cycles through the depot of minimum total length (a capacitated vehicle-routing problem, a generalisation of the TSP); (iv) schedule the cycles onto the fewest drones that complete them within the daily window (bin packing).

2. Partitioning Logic

2.1 The flight graph

Drones fly straight legs, so the natural graph is a visibility graph. Its vertices are the clients, the candidate depot sites and the obstacle corners; two vertices are joined when the segment between them crosses no restricted polygon and is at most D long. By the theorem of Lozano-Pérez and Wesley (1979) the shortest obstacle-avoiding path between two points is a polyline whose turning points are obstacle vertices. Therefore (a) two mutually visible points have the segment itself as shortest path and (b) a blocked pair is joined by a path through obstacle vertices, all of whose legs are edges of G. Dijkstra on G is consequently exact, and client-client edges are unnecessary: the graph keeps depot-client, depot-vertex, client-vertex and vertex-vertex edges, 103,976 in total for 1266 vertices, built in 2.4 s with an STRtree spatial index (O(n log n) per vertex).

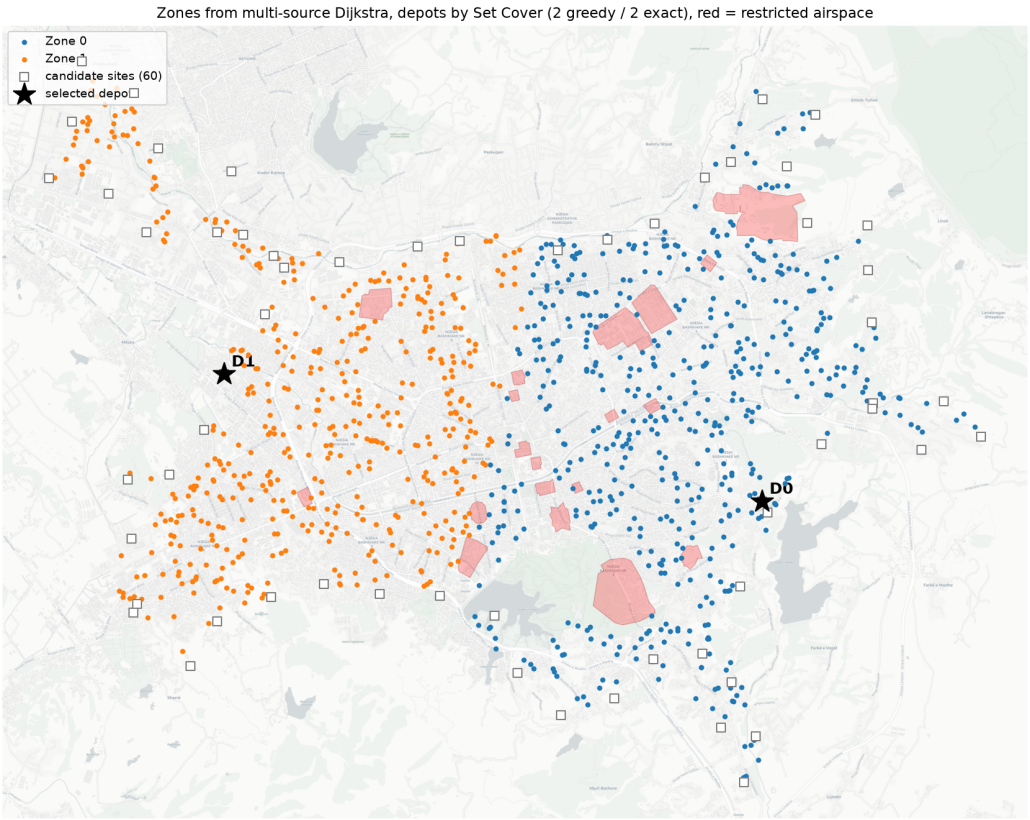


Figure 1. Built-up Tirana, the restricted airspace (red), the candidate depot sites on the outskirts (squares), the two depots selected by Set Cover (stars) and the two zones produced by multi-source Dijkstra (colours).

2.2 Candidate sites on the outskirts

A design requirement is that every depot sits on the outskirts of the city, where land is cheaper, take-off noise is tolerable and trucks can resupply. We sample 60 candidate sites every 800 m along the outer boundary of the coverage polygon (after a 400 m morphological closing so that gaps between land-use patches do not count as outskirts) and snap each to the nearest motorway, trunk, primary or secondary road within 500 m, using the OSMnx drive graph, so that depots can be resupplied by truck.

2.3 Why the zones follow from the battery, not from a grid

From every candidate a bounded Dijkstra (early exit at D/2) returns its coverage set: the clients it can serve by a feasible round trip. Set Cover (Section 3.2) selects the depots; then one multi-source Dijkstra from all selected depots labels every client with its nearest depot by shortest flight path. The zones are therefore the cells of a graph Voronoi diagram: by construction every zone lies inside the battery reach of its own depot, and each client is assigned to the depot that minimises its single-trip distance. The zone sizes are an output of the drone range: with a 12 km drone the same code needs 2 depots, with 20 km it needs 1 (Section 4.3).

Zone	Depot (UTM x, y)	Clients	Farthest client (km)	Round trip (km)	Battery used
Z0	(404179, 4574943)	518	5.45	10.89	68 %
Z1	(397229, 4576679)	482	4.16	8.33	52 %

2.4 Challenges and mitigations

Challenge	Mitigation
Several depot sets of minimum size cover all clients	p-median tie-break: among all minimum covers choose the one minimising total client-depot distance (2446 km vs 3509 km for the first greedy answer); this also balances the zones
Centroid-based depots (K-Means) land inside the city	candidates are restricted to the boundary ring; K-Means is kept only as a benchmark (Section 4.5)
Straight-line distances ignore restricted airspace	all distances are shortest paths in the visibility graph; detours are exact

Challenge	Mitigation
Overpass API outages	every OSM layer (land use, roads, restricted areas, basemap) is cached locally after the first fetch

3. Mathematical Justification

3.1 Shortest paths: Dijkstra, bounded, multi-source, A*

All distances are shortest-path distances in G computed by Dijkstra's algorithm with a binary heap in O((|V| + |E|) log |V|). Edge weights are positive lengths, so the algorithm is exact. We use three variants built on the same relaxation step: (a) bounded Dijkstra stops as soon as the smallest tentative distance exceeds D/2, which is how the coverage sets are computed; (b) multi-source Dijkstra starts with all chosen depots at distance 0 and labels every vertex with the depot it was reached from (equivalent to Dijkstra from a virtual super-source), which gives the zones in a single run instead of one run per depot; (c) A* with the straight-line heuristic h(u) = |u, target|, which is admissible and consistent because every edge weight is the Euclidean length of that edge, so A* returns the same distances as Dijkstra (verified on 60 blocked pairs) while settling 12 vertices on average instead of 737.

The distance oracle used by the routing stage answers d(i, j) = |ij| when i sees j and |ij| ≤ D (exact by the theorem) and runs a bounded Dijkstra only from vertices that have a blocked partner. On the main instance 180,933 pairs were direct and 69,891 blocked, requiring 908 Dijkstra runs. The largest detour factor (graph distance / straight line) is 2.22, the mean 1.008.

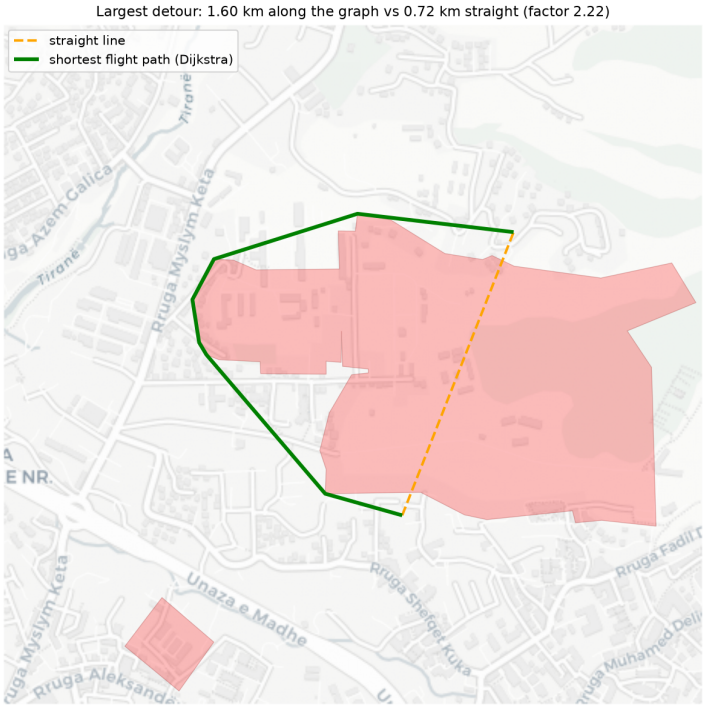


Figure 2. The pair with the largest detour: the shortest flight path (green) turns only at corners of the restricted polygon, as the visibility-graph theorem predicts.

3.2 Depot placement: Set Cover with an optimality certificate

Let U be the client set and S_j = {c : dist(j, c) ≤ D/2} the coverage set of candidate j. Choosing the fewest candidates whose sets cover U is the minimum Set Cover problem, NP-hard. Chvátal's greedy algorithm repeatedly takes the candidate covering the most still-uncovered clients; it runs in O(|S|·|U|) per iteration and uses at most H(max|S_j|) ≤ ln|U| + 1 ≈ 7.9 times the optimum. To know the real gap we also solve the 0/1 integer program min Σ x_j s.t. Σ_{j : c ∈ S_j} x_j ≥ 1 for all c with the HiGHS branch-and-bound solver of SciPy (milliseconds for 60 candidates). Result: greedy 2 depots, optimum 2, 0 unservable clients. Because several 2-depot covers exist, we enumerate all C(60, 2) combinations and keep the cover minimising total client-depot shortest-path distance (the p-median criterion), a secondary objective that shortens every single-client trip and

balances the zones.

3.3 Routing: sweep and savings on the exact metric, local search, bounds

Inside a zone, with the depot as vertex 0 and the exact distance matrix M from the oracle, the clients must be split into feasible cycles of minimum total length: a capacitated VRP, NP-hard as a generalisation of the TSP. Two polynomial constructions are used and the better one kept per zone:

- Sweep (Gillett and Miller, 1974): clients are taken in polar order around the depot and inserted into the current petal at the cheapest position (O(n) per insertion); when range or payload would be violated the petal is closed. The sweep is rotated over 12 start angles and the best run kept, which removes the arbitrariness of a single start.
- Clarke-Wright savings (1964): start from one round trip per client (always feasible by construction of the zone) and merge trips in decreasing order of the saving s(i, j) = M[0,i] + M[0,j] – M[i,j] while range and payload hold.

Every petal is improved by best-improvement 2-opt (O(1) per move evaluation) and Or-opt (segments of 1-3 clients moved, possibly reversed), then an inter-petal relocate pass moves single clients between petals when that shortens the total and both petals stay feasible; emptied petals disappear, so this pass can also reduce the number of trips. Each move only shortens a feasible cycle, so feasibility is invariant (asserted in the code). The quality of the result is measured against two lower bounds valid for any solution: trips ≥ ■Σ w_c / W_max■, and distance ≥ max(½ Σ_c (d■(c) + d■(c)), (2 / W_max) Σ_c w_c · d(0, c)), the degree-2 bound and the radial bound of Haimovich and Rinnooy Kan (1985).

3.4 Fleet sizing: bin packing

A petal of length L with n clients takes 60·L/v + n·t_service + t_turnaround minutes. Assigning petals to drones so that each drone's day fits in the 8 h window is Bin Packing. First-Fit Decreasing never uses more than 11/9 OPT + 6/9 bins; the trivial bound ■total minutes / window■ is reported beside it.

3.5 Algorithm pipeline

Step	Method	Input	Output	Complexity
1	OSM layers + buffering	city name	coverage polygon, obstacle polygons	cached
2	Monte-Carlo sampling	polygon, N	clients with payloads	O(N)
3	Visibility graph (STRtree)	clients, sites, obstacle vertices	G = (V, E)	O(|V| m log m)
4	Bounded Dijkstra per site	G, D/2	coverage sets	O(|S| (|V|+|E|) log |V|)
5	Greedy + exact Set Cover, p-median tie-break	coverage sets	depots	greedy O(|S||U|) per step; ILP
6	Multi-source Dijkstra	G, depots	zones	O((|V|+|E|) log |V|)
7	Distance oracle	zone nodes	exact matrix M	direct pairs O(1), blocked pairs Dijkstra
8	Rotating sweep / savings + 2-opt, Or-opt, relocate	M, payloads	petals	O(k n²) sweep, O(n² log n) savings
9	First-Fit Decreasing	petal durations	drones, schedule	O(p log p)

4. Implementation Results

The whole pipeline runs in about 78 s on a laptop (graph 2.41 s, depots 1.32 s, routing 72.62 s). The baseline '1 client / trip' flies every parcel on its own Depot → Client → Depot round trip.

4.1 Petal summary by zone

Zone	Clients	Method	Trips	Trips LB	km	km LB	Max clients/trip	Drones	Drones LB	Naive trips / km / drones
Z0	518	savings	124	118	674	582	6	6	6	518 / 2584 / 18

Zone	Clients	Method	Trips	Trips LB	km	km LB	Max clients/trip	Drones	Drones LB	Naive trips / km / drones
Z1	482	savings	117	111	610	528	7	6	6	482 / 2308 / 16
Total	1000		241	229	1285	1110		12	12	1000 / 4892 / 34

Multi-client routing cuts the flown distance by 74%, the number of take-offs by 76% and the fleet by 65%. The trip count is within 5.2% of the payload lower bound, so payload is the binding constraint: a trip fills up before it runs out of battery. All trips respect both constraints (Figure 5).

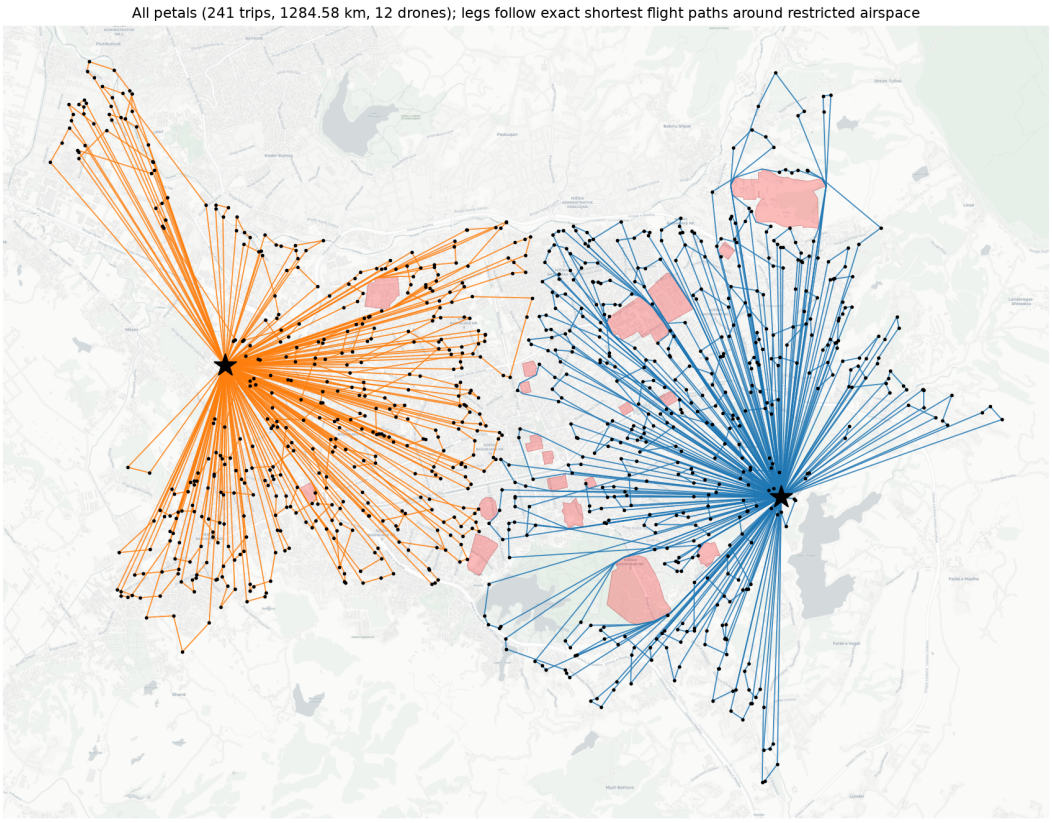


Figure 3. All 241 petals; every leg follows its shortest flight path, so the routes bend around the restricted polygons.

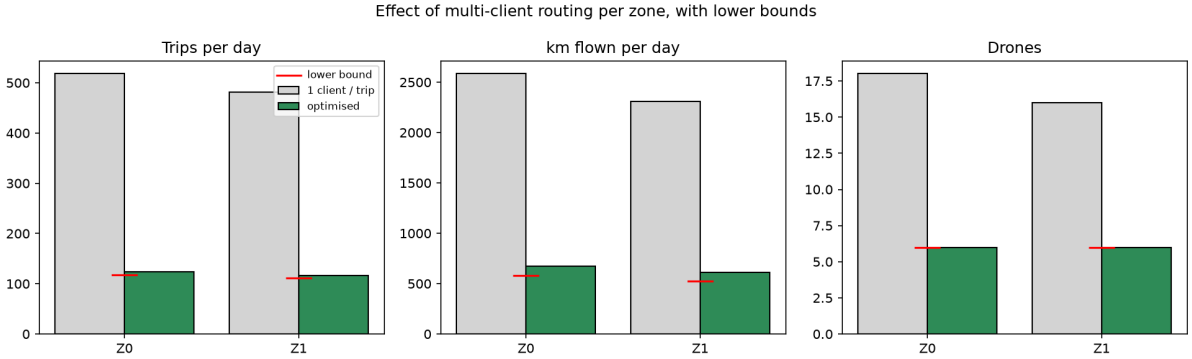


Figure 4. Trips, distance and drones per zone: single-client baseline, optimised solution, and lower bounds (red).

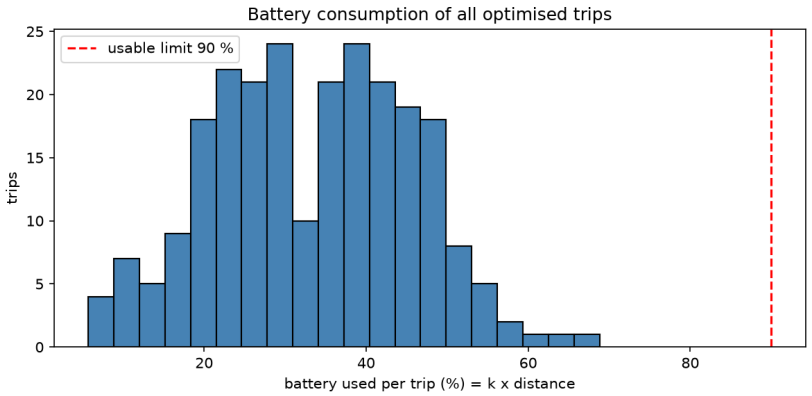


Figure 5. Battery consumed per trip (k × distance); no trip exceeds the usable limit.

4.2 Local-search ablation

On zone Z0 (518 clients) we isolate the contribution of each routing component, starting from a single-angle sweep with cheapest insertion:

Configuration	Trips	km
sweep raw (cheapest insertion)	135	911.1
sweep + NN reorder	135	911.9
sweep + NN + 2-opt	135	911.0
sweep + NN + 2-opt + Or-opt	135	911.0
rotating sweep + local search + relocate	124	781.1
savings + local search + relocate	124	674.4
lower bound	118	582.2

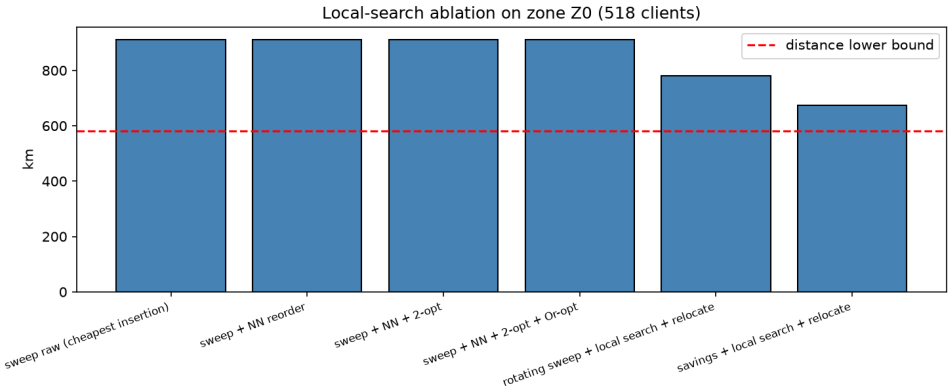


Figure 6. Ablation of the routing stage against the distance lower bound.

4.3 Sensitivity to the drone range

D_max (km)	Depots (greedy / exact)	Zone sizes	Trips (LB)	km (LB)	Drones
12	3 / 2	488, 512	241 (229)	1338 (1164)	12
16	2 / 2	518, 482	241 (229)	1285 (1110)	12
20	1 / 1	1000	239 (229)	2189 (1670)	14

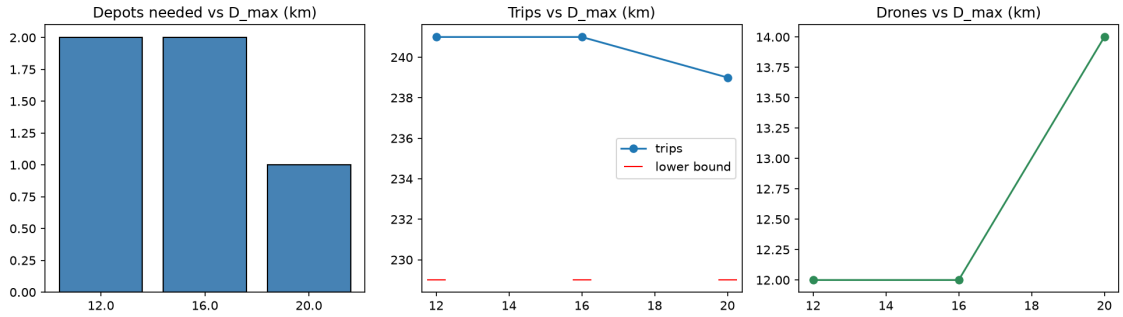


Figure 7. Depots, trips and drones as a function of the drone range.

4.4 Sensitivity to the payload

W_max (kg)	Trips (LB)	km (LB)	Max clients/trip	Drones	Best method per zone
4	371 (344)	2016 (1665)	5	16	savings, sweep
6	241 (229)	1285 (1110)	7	12	savings, savings
10	142 (138)	802 (666)	10	10	savings, savings

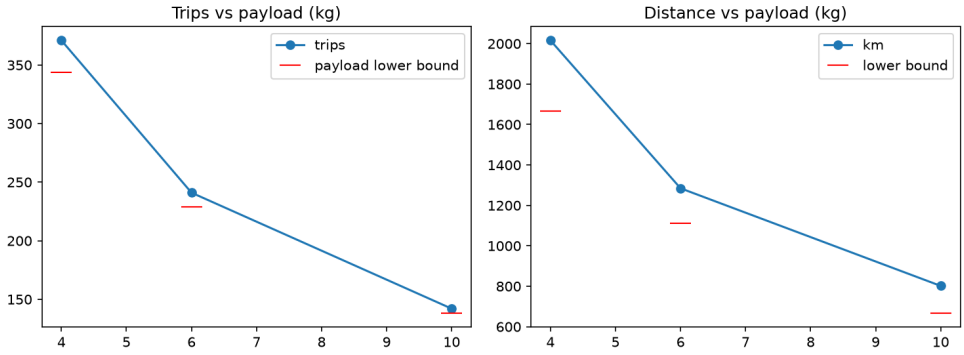


Figure 8. Trips and distance as a function of the payload limit, with lower bounds.

4.5 Depot strategy: K-Means centroids versus Set Cover on the outskirts

Strategy	Depots	Mean client-depot distance (m)	Depots inside built-up area	Metric
K-Means centroids snapped to roads (original)	4	1555	3	Euclidean, ignores restricted airspace
Set Cover + p-median on boundary sites (this report)	2	2446	0	shortest flight path, outskirts only

Centroid depots are closer to the clients on average, as expected, but they violate the outskirts requirement and are measured with straight lines that cross restricted airspace. The Set Cover solution satisfies the outskirts requirement with the provably minimum number of depots.

4.6 Scaling

Clients	|V|	|E|	Graph build (s)	Coverage + Set Cover (s)	Routing (s)	Dijkstra runs	Blocked / direct pairs
250	516	31,554	0.73	0.23	3.54	231	13,922 / 17,453
500	766	55,286	1.25	0.6	6.68	463	18,594 / 44,597
1000	1266	103,976	2.25	1.3	36.18	908	69,891 / 180,933

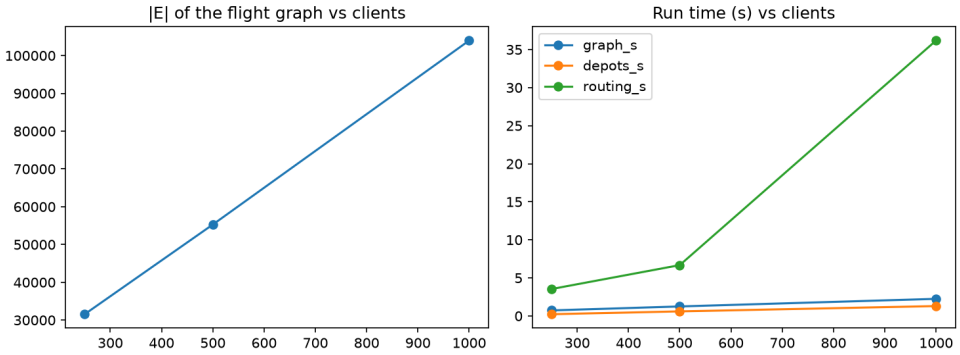


Figure 9. Graph size and run time versus the number of clients.

4.7 Interactive visualisation

main.py opens the step-through viewer: at each step one petal per zone is highlighted on the cached basemap with its length and client count, with cumulative distance and clients per zone in the title; Previous / Auto / Next buttons animate the day. Restricted areas and depots are drawn as layers, and petal geometries are the actual shortest flight paths. All static figures of this report are exported by the same module.

5. Recommendations for an operator

- Open 2 depots on the outskirts at the selected sites (west, near the Mëzez side of the ring road, and east by the artificial lake). This is the provably minimum number for a 16 km-class drone; a third depot adds no coverage and would only shorten trips that are already limited by payload.
- Start with 12 drones plus two spares (one hot spare per depot). Single-parcel flying would need 34; the routing software pays for itself before the first week.
- Payload is the binding constraint (5% above the payload bound, 16% above the distance bound). Raising W_max from 6 to 10 kg reduces trips to 142 and drones to 10; a larger battery alone would not.
- Keep the 10% reserve and treat restricted airspace as hard: the exact detours cost at most a factor 2.22 on a few legs and a mean of 0.8%.
- Operate the depots as battery-swap stations; the fleet size assumes an 8-minute turnaround.
- Re-plan daily with real orders: the pipeline accepts any client list and runs in under two minutes. Next steps are time windows, wind-dependent range, and a second routing objective (energy instead of distance).

6. Limitations and future work

- Clients are synthetic (uniform inside the built-up polygon). Real demand is clustered by district and time of day; the pipeline accepts any client list, so plugging in order data is a data change, not a code change.
- Battery consumption is linear in distance. Wind, payload mass and climb cost are ignored; a per-leg energy model would replace the edge weights without touching the algorithms.
- Restricted airspace is static and derived from OSM tags; temporary restrictions and altitude layers are not modelled.
- The sweep and savings heuristics are local-optimum methods. The reported gaps to the lower bounds (a few percent on trips, 10-15% on distance) could be narrowed with a metaheuristic such as iterated local search or an exact branch-and-price on small zones.
- Fleet sizing assumes sequential trips per drone with a fixed turnaround; a time-indexed schedule with charging queues and order time windows is the natural next step.
- Everything runs in pure Python on a laptop in about two minutes for 1,000 clients; the distance oracle is the dominant cost and would be the first thing to move to a compiled routine for city-scale demand.

Code and reproducibility

The implementation is a single Python package: config.py (parameters), src/offline_map.py and src/obstacles.py (OSM layers, cached offline), src/clients.py, src/flight_graph.py (visibility graph and distance oracle), src/shortest_paths.py (Dijkstra, bounded, multi-source, A*), src/depots.py (candidate sites, coverage, greedy and exact Set Cover, zones), src/algorithms.py and src/petals.py (routing), src/fleet.py (bin packing), src/pipeline.py (end-to-end run), src/offline_viz.py (figures and interactive viewer), with main.py, experiments.py and build_report.py as entry points. All figures and numbers in this document are produced by that code with seed 40.

References

- Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (Dijkstra ch. 24, Set Cover ch. 35).
- Lozano-Pérez, T., Wesley, M. A. (1979). An algorithm for planning collision-free paths among polyhedral obstacles. CACM 22(10).
- Hart, Nilsson, Raphael (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE TSSC 4(2).
- Chvátal, V. (1979). A greedy heuristic for the set-covering problem. Mathematics of Operations Research 4(3).
- Gillett, B., Miller, L. (1974). A heuristic algorithm for the vehicle-dispatch problem. Operations Research 22(2).
- Clarke, G., Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. Operations Research 12(4).
- Croes, G. A. (1958). A method for solving traveling-salesman problems. Operations Research 6(6).
- Haimovich, M., Rinnooy Kan, A. H. G. (1985). Bounds and heuristics for capacitated routing problems. Math. of OR 10(4).
- Johnson, D. S. (1973). Near-optimal bin packing algorithms. PhD thesis, MIT.
- OpenStreetMap contributors; Boeing, G. (2017). OSMnx. Computers, Environment and Urban Systems 65.